



#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Non-Invasive AI Agent Observability with OBI

See every AI agent call — without changing a single line of code

Haibin Zhang · Alibaba Cloud Observability

OpenTelemetry eBPF Instrumentation (OBI)

Agenda

01

What is OBI & How It Works

eBPF-based, zero-code observability agent for OpenTelemetry

02

Why Agents Break APM

Longer chains, finer evidence, SDK blind spots, sandbox opacity

03

In Action

Real cases: RAG, tokens, sandbox

04

AI Agent Observability

4-layer model, key metrics, dashboard & best practices



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE



China 2026

01 — What is OBI



OBI: A Kernel-Level Lens for Observability

OTel-native, born from Grafana Beyla, now an OpenTelemetry project.

OBI uses Linux eBPF to intercept & parse app network traffic in the kernel — standard OTel Traces & Metrics with ZERO code, ZERO SDK, ZERO restart.

Application Observability

Traces + RED metrics across
Web, DB, MQ, GenAI

Network Observability

L3/L4 flows, TCP RTT,
service topology & health

Log Enrichment

Auto-inject trace_id for
Trace↔Log correlation

OBI at a Glance

WHAT IS OBI

24

protocols
instrumented

20+

GenAI providers
& MCP

5

languages deep-
instrumented

5.8+

min Linux kernel
(4.18 RHEL)

0

lines of code
changed

Standard OTLP export → any OpenTelemetry-compatible backend. Deployed as a K8s DaemonSet with auto service discovery.

HTTP/1.1 · HTTP/2 · gRPC · GraphQL · MySQL · PostgreSQL · Redis · MongoDB · Kafka · MQTT · NATS · DNS · GenAI ...



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Protocol Coverage Panorama

Web & RPC

HTTP/1.x · HTTP/2 · gRPC · GraphQL
TLS decryption via uprobe
Frame-level HPACK decode

Databases & MQ

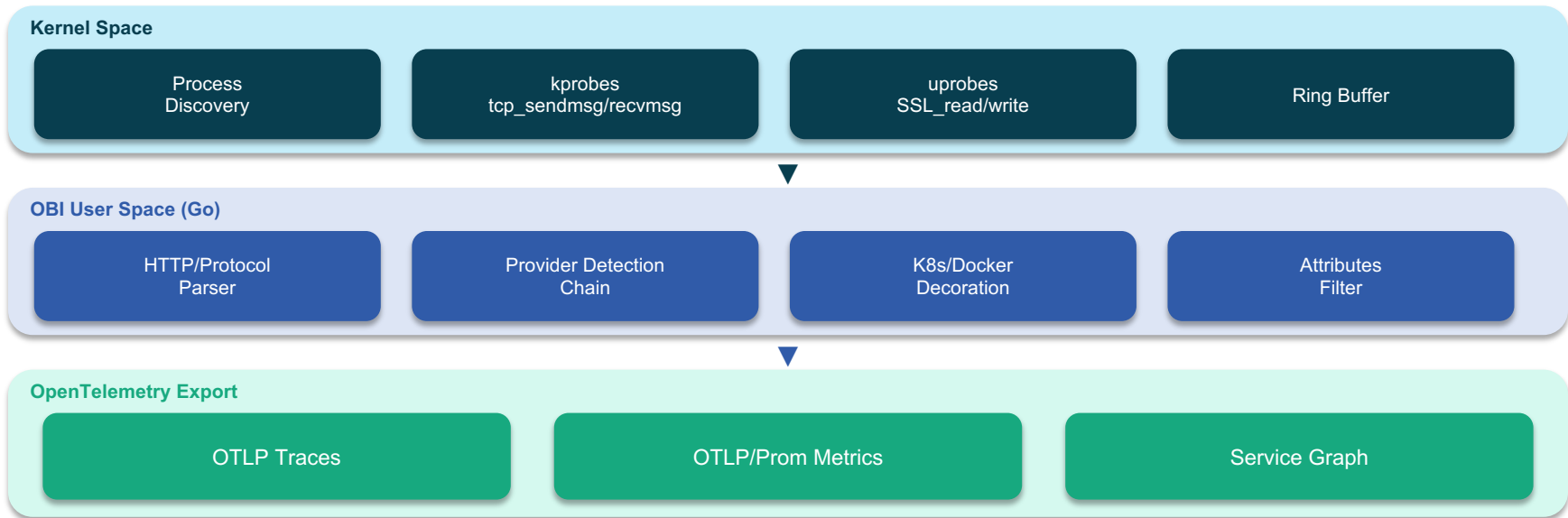
MySQL · PostgreSQL · Redis · MongoDB
Kafka · MQTT · NATS · SQS
Auto SQL text & topic extraction

GenAI & GPU

OpenAI · Anthropic · Gemini · Qwen
Ollama · Rerank · Vector DBs · MCP
CUDA kernel launch & memory ops

Three-level waterfall matching: kernel annotation → deterministic match → heuristic fallback.

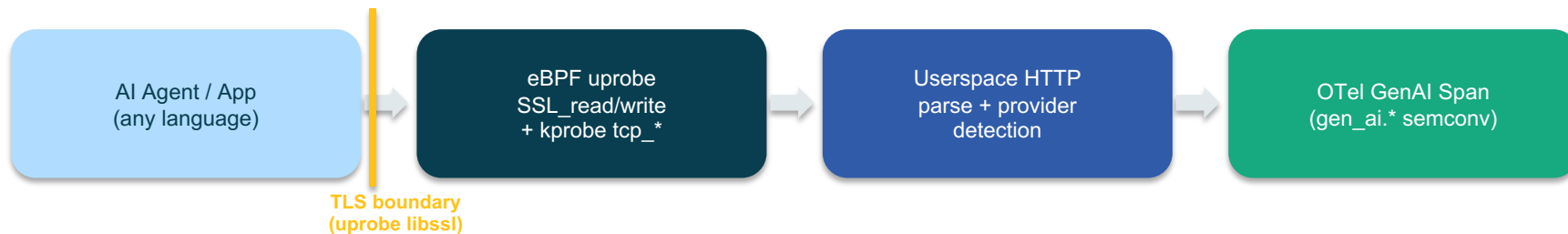
How OBI Works — Kernel Bytes → OTel Spans



A per-executable eBPF tracer feeds an explicit user-space DAG; disabled features are physically removed.

The Non-Invasive Data Path

Encrypted Traffic → GenAI Span



- Captures plaintext BEFORE encryption / AFTER decryption — no keys, no MITM, no LD_PRELOAD.
- Large-buffer stitching reconstructs full 10KB+ JSON LLM bodies from ring-buffer segments.
- Language-agnostic: any app over TCP + OpenSSL/BoringSSL is instrumented identically.

Protocol-Aware GenAI Detection

11 GenAI subtypes auto-detected

OpenAI · Anthropic · Gemini
AWS Bedrock · Qwen/DashScope
Ollama · MCP · Embedding
Rerank · Vector Retrieval
OpenAI-compatible gateways

URL path

Response headers

Body shape

Provider resolved

Semantic-convention attributes emitted

gen_ai.request.model
gen_ai.response.model
gen_ai.operation.name
gen_ai.provider.name
gen_ai.usage.input_tokens
gen_ai.usage.output_tokens
gen_ai.tool.name
mcp.method.name
mcp.session.id

Also emits `gen_ai.client.operation.duration` + `gen_ai.client.token.usage` metrics for latency & cost tracking.

GenAI Semantic Conventions & Metrics

Model & Operation

gen_ai.request.model
gen_ai.response.model
gen_ai.operation.name
gen_ai.provider.name
gen_ai.response.finish_reasons

Token Usage & Cost

gen_ai.usage.input_tokens
gen_ai.usage.output_tokens
gen_ai.usage.cache_read
Normalized: OpenAI
prompt/completion,
Anthropic usage.*, Ollama eval_count,
Gemini usageMetadata.*

Tools / MCP / Content

gen_ai.tool.name / type
gen_ai.tool.call.arguments / result
mcp.method.name / session.id
Optional: gen_ai.input_messages
gen_ai.output_messages capture

RED metrics: `gen_ai.client.operation.duration` + `gen_ai.client.token.usage` for latency & cost dashboards.

Cross-Language Context Propagation

Goroutines drift across OS threads; asyncio runs thousands of coroutines per thread. OBI rebuilds causality in the kernel.

Go goroutines

Hook runtime.newproc1 / casgstatus;
walk up to 6 parent-G levels to find
the inbound trace context.

Python asyncio

4 uprobes: task_step, Task.__init__,
PyContext_CopyCurrent, context_run
— covers await/create_task/gather.

Cross-process (tpinjector)

Kernel injects traceparent 3 ways:
HTTP/1 header (sk_msg), HTTP/2
HPACK, TCP Option kind=25.

Beyond Applications: GPU, Network & Logs

GPU / CUDA

uprobe libcuda:
cudaLaunchKernel (grid/block)
cudaGraphLaunch
cudaMalloc / cudaMemcpyAsync
Shared forwarder with net spans.

Network Flows

TC hooks: L3/L4 IPv4/IPv6+TCP/UDP
K8s metadata / reverse DNS / GeoIP
TCP RTT & connection-failure
congestion delimiting.

Log Enrichment

kprobe tty_write / pipe_write
intercepts JSON logs & injects
trace_id + span_id in place.
Zero app code changes required.

Deploy in Kubernetes

DaemonSet: one OBI pod per node

Service discovery via
discovery.instrument:
exe_path / open_port /
k8s_namespace / labels

Auto K8s decoration:
pod / namespace /
deployment / node

k8s-cache (optional)

Independent gRPC service

Centralized K8s informer cache

Lowers API server pressure

Less memory per OBI pod

Faster cold-start in large clusters

Image: otel/ebpf-instrument · Privileged DaemonSet · Needs BPF + SYS_ADMIN + NET_ADMIN caps.

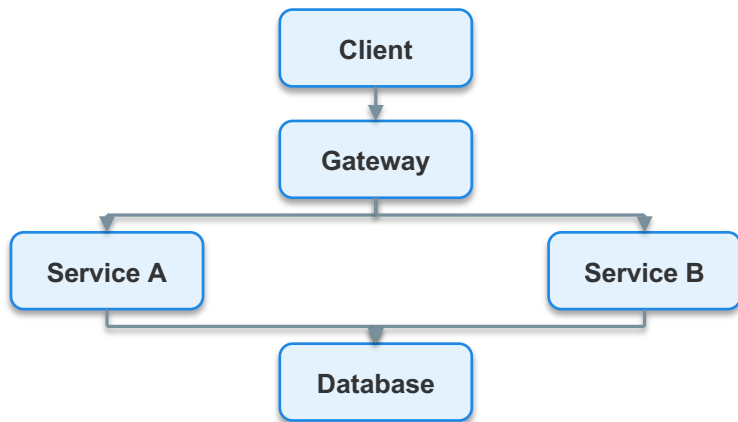
02 — Why AI Agents Break Traditional Observability



The New Shape of an AI Request

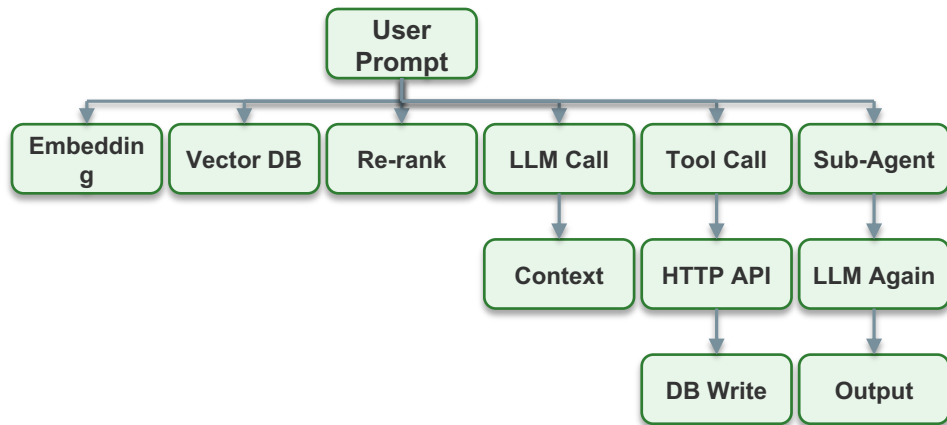
One prompt fans out into a whole call tree across services and providers.

Traditional Web Service



Depth 3 · Static topology · Predictable

AI Agent



Depth 4+ · Dynamic fan-out · Every request differs

Your APM covers the left tree well. OBI fills the gaps in the right tree.

Four Reasons You Need Non-Invasive Observability

Longer chains

One request spans multiple services, models and providers. Thread/PID correlation falls apart.

Finer-grained evidence

HTTP 200 isn't enough. You need model, tokens, similarity score, tool name to locate issues.

SDKs fall short

Raw HTTP calls have no SDK object to hook. Monkey-patch libraries silently do nothing.

Sandbox execution needs a boundary-aware design
In-container process and I/O evidence needs an agent in the kernel domain being observed.

Traditional APM/SDK vs. OBI

Dimension	Traditional SDK/APM	OBI (eBPF)
Code changes	Manual SDK integration	Zero — kernel-level
Language support	Per-language SDK needed	Any language over TCP
Deployment	Rebuild & restart	DaemonSet, no restart
Runtime overhead	In-process SDK cost	eBPF-based, low overhead
Coverage	Only instrumented paths	All network traffic
TLS visibility	Needs proxy or agent	uprobe on libssl.so
Raw-HTTP agents	Invisible	Fully captured
Sandbox / VM boundary	Usually a black box	runc node; microVM guest OBI*

* Issue #3210 proposal: opt-in Process & I/O. Kata/Firecracker need guest-side OBI; gVisor is unsupported.



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

OBI and otelc — Complementary Layers

otelc — Compile-Time (Go)

- Baked into Go binary at compile
- Deep in-process spans
- Go only
- Needs build-pipeline access
- No runtime privileges needed

OBI — Runtime (eBPF)

- Runs alongside app, observes via eBPF
- Protocol-level spans (HTTP/gRPC/DB)
- Polyglot (Go, Java, Python, .NET, ...)
- No code or build change
- Needs Linux kernel 5.8+ and privileges

Use them together — same Go service:
otelc for business logic · OBI for network and non-Go components

Source: opentelemetry.io/docs/zero-code/go/choosing-auto-instrumentation (PR #11035) — official OTel decision guide.



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE



China 2026

03 — AI Agent Observability in Action



The 4-Layer AI Agent Observability Model

Layer 1 — GenAI Semantics

LLM calls: model, tokens, provider
Standard gen_ai.* attributes
Multi-provider normalization

Layer 2 — Tool & MCP Calls

Tool name, type, arguments, result
MCP session + method tracking
Agent planning → action topology

Layer 3 — RAG & Retrieval

Embedding latency + vector scores
Rerank pass-through detection
Namespace + corpus validation

Layer 4 — Infrastructure & Runtime

Network + TLS evidence
Proposed: process tree, stdio bytes, file I/O

OBI supplies non-invasive network evidence today; Issue #3210 proposes opt-in Process & I/O runtime signals.

What to Measure — Key Metrics & Thresholds

Dimension	What to Measure	OBI Attribute	Alert Threshold
LLM Performance	Time To First Token, inference p99	gen_ai.client.operation.duration	TTFT > 2s
Token Economics	Input / output tokens, cost per query	gen_ai.usage.input_tokens	MoM growth > 50%
Tool Execution	Tool call success rate, avg duration	gen_ai.tool.name + duration	error rate > 5%
RAG Quality	Vector similarity score, hit count	db.operation + score attrs	score < 0.5
Reliability	Error rate by provider and model	gen_ai.response.finish_reasons	4xx/5xx > 2%
Runtime Execution*	exec/fork/exit, stdio bytes, file ops	process.* + OBI I/O attrs	unexpected exec / path

* Proposed: reuse process.* where applicable; selector + sampling bound the event volume.

Case 1 — 'Answers Off the Mark' (RAG)

APM shows one /chat/completions call: 2.8s, HTTP 200.

Embedding (text-embedding-3-small)

120ms ✓

Vector Search (Top-K=5, ns=docs-v2)

score=0.31 ⚠

Rerank (1 doc in)

no-op ⚠

LLM GPT-4o (2100 in / 450 out)

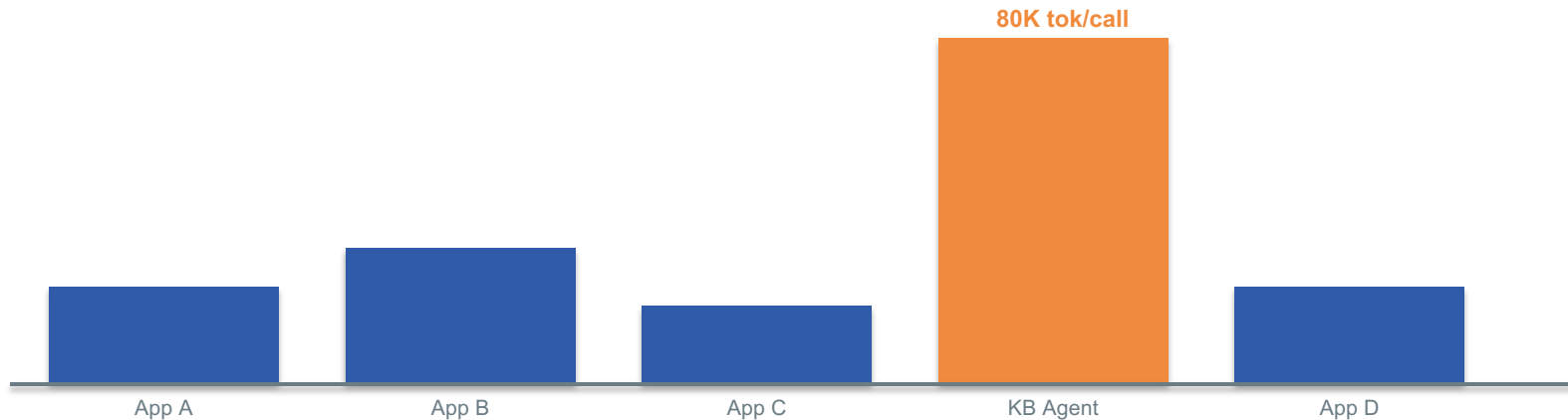
context starved

Root cause: vector namespace misconfig (docs-v3 vs docs-v2) — not model hallucination.

Fix → score 0.82+. Action: version-check namespaces; alert on hit-count<3 or score<0.5.

Case 2 — Token Bill Up 320%

AWS Bedrock input tokens +320% MoM.

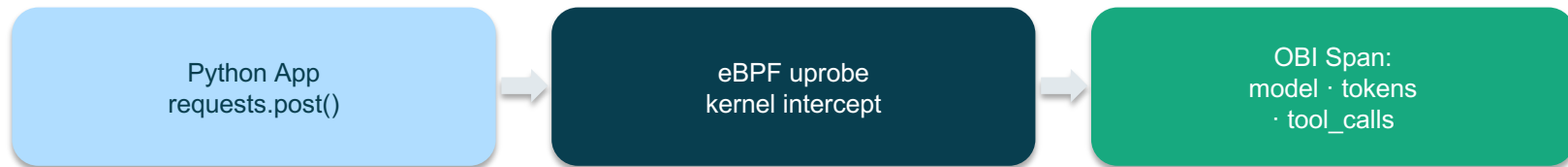


One KB agent stuffing a 200-page PDF into every system message.

Action: chunk & index; cap system-message tokens; per-app token alerts.

Case 3 — Raw HTTP Calls to an LLM

Python requests calling Qwen /chat/completions — no SDK. All monkey-patch instrumentation does nothing.

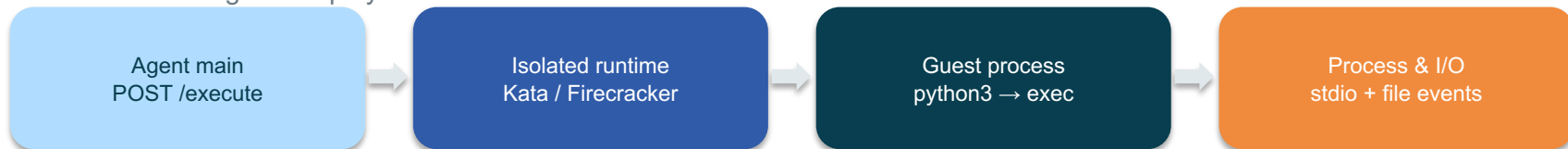


- Provider auto-identified as qwen (via X-DashScope-Request-Id + Host).
- Same dashboard, identical fields as SDK-instrumented apps.

OBI covers every app that calls an LLM over HTTP — not just SDK-instrumented ones.

Case 4 — Guest-Side Visibility

AI coding agents run model-generated code in isolated runtimes. The host sees an API call; guest process and I/O evidence needs guest deployment.



Proposal #3210: guest-kernel OBI reports exec lifecycle, stdio bytes, and file I/O.
Host-level OBI cannot cross a VM boundary; gVisor is unsupported.

Roadmap — Process & I/O Proposal

1. Scope the signals

exec/fork/exit process tree; stdin/stdout/stderr byte counts; open/read/write.

2. Emit useful data

Aggregate metrics + structured events; reuse OTel process.* with OBI attributes.

3. Control the data

Independent opt-in selector; PID filters, path-prefix allowlists, sampling.

4. Deploy per runtime

runc: node DaemonSet. Kata/Firecracker: guest OBI.

Alibaba Cloud Observability Community

Scan to join our DingTalk groups — contribute, test, and build together



Go

LoongSuite Go Agent



Python

LoongSuite Python SIG



Java

LoongSuite Java SIG



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

GitHub: github.com/NameHaibinZhang

Key Takeaways & Resources

- ✓ Zero-code, zero-SDK network observability in supported kernel runtimes — instrument outside the app.
- ✓ GenAI-aware today; Issue #3210 proposes opt-in process, stdio, and file I/O signals.
- ✓ Runtime boundary matters: runc node-level; Kata/Firecracker guest-side; gVisor unsupported.

Resources

GitHub: github.com/open-telemetry/opentelemetry-ebpf-instrumentation

Proposal: Issue #3210 · OpenTelemetry project · CNCF

Docs: opentelemetry.io · eBPF: docs.ebpf.io



China 2026

Thank You

Questions? · Non-Invasive AI Agent Observability with
OBI

